A Multi-Agent Interaction

CS404/924 Agent Based Systems – Coursework 2025-26

1 Introduction

Imagine an auction of paintings by four famous artists: Picasso, Van Gogh, Rembrandt and Da Vinci. The auction proceeds in rounds. In each round, the auctioneer presents a piece to be sold. All bidders then write their bids for the item on a secret sealed note that is handed to the auctioneer. The highest bidder wins the piece and pays their bid ("first-price auction"). Then, the auctioneer starts the next round with a new painting. The auction continues until there are no more paintings to sell. The overall winner is then decided by a winning condition that is specified below.

The objective is to implement strategies for a Python bidding bot that will participate in this game.

2 The Game

This is an extensive game with simultaneous moves encoded as an auction played sequentially. The game runs in the following way. An Auctioneer initialises a room full of bots, then sets up the auction. This involves giving each bot the same budget to spend and setting the sequence of items (and their types) to be sold.

The Auctioneer will then sell the paintings one after another. In each round, the Auctioneer will announce the item type (Picasso, Van Gogh, Rembrandt, or Da Vinci) to be bid upon and ask the bots for bids. Your bot will use the information that the Auctioneer gives, including the outcomes of the previous rounds, to determine an amount to bid. Your bid cannot exceed your remaining budget. Once all bots have bid, the Auctioneer will declare the highest bidder the winner, who will then be charged a payment equal to their bid and receive the item. If the top bids draw, then the winner is chosen at random from those bidders. For each round, there is exactly one winner.

The starting budget for every bidder is 1001. The auction will continue for 200 rounds. Any remaining unspent budget at the end of the auction is not considered in determining the winner.

Winning condition: At the end of the auction, each bidder's collection will be evaluated based on the number of paintings they own from each artist. For each bidder, the product of the counts of paintings they hold from each artist will be calculated (i.e., multiply the number of paintings from Artist 1 $\times$ Artist 2 $\times$ Artist 3 $\times$ Artist 4). The bidder with the highest product wins.

For example, Player A has paintings (3, 3, 1, 1) with a score of 9, Player B has paintings (2, 2, 2, 2) with a score of 16, and Player C has paintings (10, 1, 1, 0) with a score of 0. Here, Player B would be declared the winner.

You will write your strategies in your bot. Your bot will be tested in a series of different auction rooms against bots of varying difficulty and number. Finally, your bots will be tested against each other in a tournament.

3 Implementation

Provided to you are two main Python 3 files to run the auctions: `auctioneer.py` and `arena.py`. We also provide you with some example bots in the bots folder, `random_bot.py`, `flat_bot_10.py` and `u1234321.py`.

3.1 auctioneer.py

`auctioneer.py` contains the definition for the auctioneer class, which sets up and runs the auction. We will use this exact same file while marking your bots, so DO NOT CHANGE ANY OF THE CODE IN THIS FILE.

It has the following arguments:

- `room`(list of modules): A "room" is a list of bots that will play the auction. The bots are module objects. There is an example of how to import them at the top of the `auctioneer.py` file, and how to pass them to the auctioneer as a list at the bottom of the `auctioneer.py` file. There are also examples in the `arena.py` file.
- `painting_order`(list of strings): A list of the sequence of painting types that will be auctioned in each of the rounds. If this is set to *None*, then a random order will be used.
- `slowdown`(float): How long to wait at each round of the auction. If you set this to zero the auctions will run fast.
- `verbose`(boolean): Whether the auctioneer prints updates to the terminal or not.
- `output_csv_file`(string): The auctioneer automatically logs the result of every round in an auction. This defaults to 'data/auctioneer_log.csv', but you can specify a different filename.

When the auctioneer is initialised it will automatically set up the auction and all the bots in the room with the `initialise_bots()` method.

The method `run_auction()` will run the auction until it is completed.

It is not necessary to know how the auctioneer class works to do well on the coursework. But if you are interested there are more details in the `README.md` file.

3.2 arena.py

The `arena.py` file is provided as a convenient way to run auctions. This includes some methods that show examples of how to run auctions. This is given to you as an example, so please feel free to change any of the code here and experiment.

You might want to run an auction slowly, with a full print out to the terminal of the auction's progress, as shown in `run_basic_auction()`. This will be useful to see how your bot is performing live. Or you might want to run lots of auctions quickly, as shown in `run_lots_of_auctions()`.

3.3 bots

In the bots folder we included a few bots for you to practice with. We have given you 3 bots. `bots/u1234321.py` is a good starting point for your own bot, with everything commented clearly. `bots/flat_bot_10.py` and `bots/random_bot.py` are example bots that should be easy to beat.

3.4 bots/u1234321.py

`u1234321.py` is an example bot that you can use as a starting point to write your own bot.

Each bot is a class, with one main method, `get_bid()`, that allows you to make bids on paintings. It is in this method that you should implement your strategy. The comments on the method explain what each argument is. Currently, the strategy just returns a random integer between 0 and the amount of budget that your bot has left.

Your bot will be initialised at the start of the auction, and then will play until the auction finishes. This means that your bot can hold internal state variables during the auction, which may be useful for some strategies. You can add these variables, or any other code, in the `__init__()` method, if you want to. Please change the self.name="1234321" variable to your own ID number.

Feel free to add any extra methods that you might need, but be careful that the main method keeps the same input variables and returns an integer bid without crashing.

If your bot crashes then your bid will be set to zero. If your bot returns a float then this will be rounded down to an integer. If your bot bids more than their budget then their bid will be set to zero. If your bot takes longer than 3 seconds to return a bid, then again your bid will be set to zero.

3.5 bots/random_bot.py and bots/flat_bot_10.py

We've included some simple bots for you to play against.

- `random_bot.py`: Bids a random integer between 0 and the bot's remaining budget.
- `flat_bot_10.py`: Bids 10 on everything.

You can add your own. You can see how to import bots and play against them in the `arena.py` file.

4 Submission

Your coursework submission will consist of a single compressed file (either .zip or .tgz) containing your bot in a python file and your writeup as a pdf. The coursework file should be submitted through Tabula.

4.1 Your Bot

Please save this as `u<YOUR WARWICK ID NUMBER>.py`. The main class should be called Bot, and in the `__init__()` method you should set `self.name = <YOUR WARWICK ID NUMBER>`. Here is an example:

`u1867321.py`

```
class Bot(object):

    def __init__(self):
        self.name = 1867321
        ....

    def get_bid(self, ....):
    ....
```

We recommend that you can use `bots/u1234321.py` as a starting point.

Here is an example of how your bot will be imported and run:

```
from auctioneer import Auctioneer
from bots import random_bot
from bots import u1867321 # Your id number here


room = [random_bot, u1867321]
game = Auctioneer(room=room, slowdown=0)
winner = game.run_auction()
print("The winner is ", winner)
```

Please make sure that your bot will run in this way, replacing the id number here with your own id number. If you run the above script you should see the auction run very fast and then print out "The winner is [1867321]", with your id number shown (if your strategy can beat a `random_bot`).

You can test that your bot runs correctly by using the functions provided in the arena.py file. If your bot crashes you will lose marks!

4.2 Your Writeup

Please save this as <YOUR WARWICK ID NUMBER>.pdf. Alongside the code submission, you are required to write a pdf report, of **at most** three pages including references, of the theory to support your strategies. It should explain the reasoning behind your strategy design including evidence of relevant theory and/or testing. The pdf should be written in IEEE two-column conference format. You are free to design and think about your strategies in your own unique way, and we encourage you to use what you have learnt from the course.

5 Evaluation

The coursework is worth 25% of the module credit. Its marking scheme is structured as follows:

- $\frac{1}{2}$ of the total mark: Strategy performance against various bots including other submissions.
- $\frac{1}{2}$ of the total mark: Quality of the written report (how well you explain and analyse your strategies).

Your bot will be tested in a range of different auction "rooms", made up of bots of different difficulties, in different room sizes and compositions. For example, you will be tested in a room against one `random_bot`. You will also be tested in a room with a mix of up to 10 bots with different strategies. Your bot's performance in this set of rooms will form the majority of your mark for strategy performance.

We will also play the student bots against each other, and this will contribute to your mark for strategy performance.

6 Cheating/Plagiarism

This coursework is your own work. Group submissions are not allowed. All submissions will be put through plagiarism detection software which compares against a number of sources, including other submissions for CS404, submissions at other universities, web sources, conference papers, journals and books. Please see the student handbook for more information, including the policy regarding the use of genrative AI tools:

`https://warwick.ac.uk/fac/sci/dcs/teaching/handbook/coursework`.

In addition, please note that any attempt to access other bots, modify the auctioneer class or otherwise intervene in the fair running of the auctions will be considered cheating and sanctioned accordingly.

Acknowledgements

The code was developed by Charlie Pilgrim (`charlie.pilgrim@warwick.ac.uk`), Department of Mathematics, University of Warwick. A previous version of the coursework, from which this takes inspiration, was written by Alexander Carver, Department of Computing, Imperial College London. Further precious input for the coursework came from Charlotte Roman, Department of Mathematics, University of Warwick.